

AI Complaint

Klasifikasi dan penentuan prioritas keluhan toilet berbasis LLM, dengan verifikasi foto bukti

UPI Kampus Tasikmalaya · [ai-complaint.yvrtz.workers.dev](#) · [github.com/rdnsa/AI-complaint](#)

Daftar isi

1. Ringkasan sistem

2. Tumpukan teknologi dan pembagian kerja dua model

3. Arsitektur dan struktur proyek

4. Model data: tabel, kolom, migrasi, seed

5. Lokasi dan kode QR

6. Alur laporan dari ujung ke ujung

7. Analisis LLM teks

8. Verifikasi foto bukti (LLM vision)

9. Akun, sesi, dan wewenang

10. Jejak audit

11. Ringkasan harian, statistik, dan grafik
12. Papan publik dan papan peringkat

13. Penyimpanan foto

14. Referensi API lengkap

15. Antarmuka pengguna

16. Konfigurasi, variabel, dan rahasia

17. Pengembangan lokal

18. Penyebaran dan operasi

19. Pemantauan dan pemecahan masalah

20. Keputusan rancangan yang berbiaya

21. Batasan yang diketahui dan pekerjaan lanjutan

22. Lampiran: contoh respons, glosarium

1. Ringkasan sistem

Mahasiswa memindai QR di pintu toilet, menulis keluhan dengan bahasa bebas, lalu *pretrained LLM* mengubahnya menjadi tiket kerja terstruktur — kategori, prioritas, ringkasan netral, dan rekomendasi tindakan. Petugas kebersihan mengerjakannya dari dashboard, dan hanya dapat menutup laporan dengan foto bukti yang **dinilai bersih oleh model vision**. Tidak ada pelatihan model dan tidak ada dataset yang perlu dilabeli; kendali mutu dilakukan lewat rekayasa prompt dan validasi keluaran.

```
QR per lantai → pilih jenis WC → tulis keluhan + foto → LLM teks (DeepSeek)
                                     ↓
                                dashboard petugas + cron 17.00 WIB → ringkasan harian
                                     ↓
petugas unggah foto bukti → LLM vision (Gemini): bersih? → selesai
```

Fitur inti (kontribusi penelitian)	Fitur pendukung
1. Klasifikasi otomatis ke enam kategori, boleh lebih dari satu 2. Penentuan prioritas rendah/sedang/tinggi berbasis risiko keselamatan 3. Ringkasan harian satu paragraf, otomatis pukul 17.00 WIB 4. Papan peringkat pelapor beserta jumlah yang benar-benar selesai 5. Verifikasi foto bukti: benar toilet, dan sudah bersih?	• Papan laporan publik, dapat dibaca siapa saja • Foto keadaan wajib dari pelapor; foto bukti wajib dari petugas • Jejak audit yang tidak dapat dihapus dari aplikasi • Dashboard: KPI, deret harian, komposisi prioritas, pola hari×jam, gedung×kategori, waktu penyelesaian • Pelacakan status untuk pelapor tanpa akun • Peta gedung di halaman depan; antarmuka dwibahasa ID/EN

2. Tumpukan teknologi

Lapisan	Pilihan	Alasan
Bahasa	TypeScript 5, mode <code>strict</code>	Binding D1 dan R2 hanya tersedia di runtime JS/TS Workers
Runtime	Cloudflare Workers, <code>nodejs_compat</code>	Tanpa server, mulai dingin di bawah 20 ms, gratis pada skala kampus
Framework API	Hono 4	Router ringan yang berjalan native di Workers; middleware cookie dan JWT bawaan

Validasi	Zod 3	Satu skema untuk body HTTP dan untuk keluaran LLM
Basis data	Cloudflare D1 (SQLite)	Binding langsung, migrasi ter-versi, <code>json_each</code> untuk kolom kategori
Penyimpanan berkas	Cloudflare R2	Diakses lewat binding, disajikan ulang oleh Worker
Frontend	React 18 + Vite 6 + Tailwind 3, react-router 6	Satu Worker melayani API dan aset statis, sehingga tanpa CORS
i18n	Kamus buatan sendiri (<code>web/src/lib/i18n.tsx</code>)	Bahasa Indonesia mendefinisikan bentuk kamus; TypeScript menolak kunci yang belum diterjemahkan
LLM teks	DeepSeek <code>deepseek-chat</code>	JSON mode, latensi ~1,1 detik, biaya rendah; tidak bisa membaca gambar
LLM vision	Google Gemini <code>gemini-3.6-flash</code>	Menilai foto bukti; endpoint kompatibel OpenAI sehingga penyedia dapat diganti tanpa mengubah kode
Dokumen	HTML + Chrome headless → PDF	Satu sumber untuk layar dan cetak (<code>npm run docs</code>)

Pembagian kerja dua model

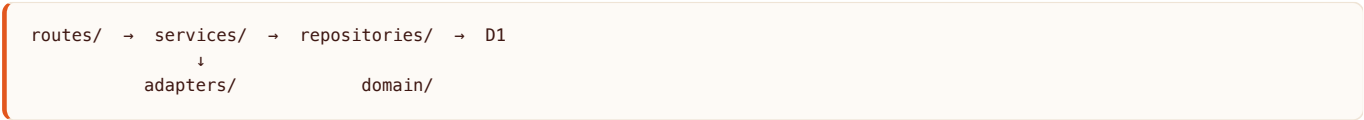
Kedua model tidak pernah dipanggil untuk pekerjaan yang sama. Pemisahannya mengikuti jenis masukan: teks ke DeepSeek, gambar ke Gemini.

	DeepSeek <code>deepseek-chat</code>	Gemini <code>gemini-3.6-flash</code>
Masukan	Teks keluhan mahasiswa; daftar ringkasan sehari	Foto bukti petugas + teks keluhan asli sebagai konteks
Pekerjaan	(1) Klasifikasi kategori, (2) prioritas, ringkasan, rekomendasi, (3) ringkasan harian	(5) Menilai foto bukti: benar toilet? sudah bersih?
Kapan dipanggil	Tiap laporan masuk (asinkron, setelah respons dikirim) dan sekali sehari lewat cron	Tiap petugas menekan “Tandai selesai” (sinkron; petugas menunggu putusan)
Parameter	<code>temperature 0.1</code> , <code>max_tokens</code> 500 (analisis) / 700 (ringkasan), <code>response_format json_object</code> , <code>timeout</code> 30 detik	<code>temperature 0.1</code> , <code>max_tokens 2000</code> , gambar <code>detail: low</code> , <code>timeout</code> 30 detik
Bila gagal	Laporan tetap tersimpan tanpa label (<code>ai_status='gagal'</code>), dapat dianalisis ulang	Penutupan ditolak (502); petugas diminta mencoba lagi
Fungsi di kode	<code>analisaKeluhan()</code> , <code>ringkasHarian()</code>	<code>periksaFotoBukti()</code>
Konfigurasi	<code>LLM_BASE_URL</code> , <code>LLM_MODEL</code> , <code>LLM_API_KEY</code>	<code>VISION_BASE_URL</code> , <code>VISION_MODEL</code> , <code>VISION_API_KEY</code>
Hasil disimpan di	<code>reports.kategori</code> , <code>prioritas</code> , <code>ringkasan</code> , <code>rekomendasi</code> , <code>ai_*</code> ; <code>daily_summaries</code>	<code>reports.bukti_ai_*</code> ; <code>aktivitas</code> (<code>bukti_ditolak</code> , <code>verifikasi_gagal</code>)
Biaya	Berbayar, sangat murah	Tingkat gratis Google AI Studio

Keduanya dipanggil lewat satu fungsi yang sama, `chatJSON()` di `src/adapters/llm.ts`, karena kedua penyedia memakai format *chat completions* OpenAI. Yang berbeda hanya tujuan (URL, model, kunci) dan isi pesan.

3. Arsitektur dan struktur proyek

Backend disusun berlapis dengan arah ketergantungan hanya ke dalam.



Lapisan	Tanggung jawab	Aturan yang dipatuhi
domain/	Kosakata (enum peran, kategori, prioritas, status, hasil bukti), entitas, pemetaan DTO, aturan penutupan	Tidak mengimpor apa pun — tanpa Hono, D1, R2
repositories/	Seluruh pernyataan SQL, satu berkas per agregat	Tahu basis data, tidak tahu HTTP
services/	Use case dan aturan bisnis	Tidak menyebut Hono sama sekali
adapters/	LLM (teks + vision), R2, hash sandi, sesi JWT, konversi waktu WIB	Membungkus dunia luar di balik antarmuka kecil
routes/	Parse, validasi, delegasi, format respons	Nol SQL, nol aturan bisnis
index.ts	Composition root: pasang route, fallback SPA, penanganan cron	Satu-satunya berkas yang melihat semua lapisan

Konsekuensi yang dapat diuji: seluruh pernyataan SQL berada di `repositories/`. Berpindah dari D1 ke Postgres berarti menulis ulang satu direktori, bukan seluruh berkas route. Aturan “laporan hanya boleh ditutup bila ada bukti yang lolos verifikasi” tinggal di lapisan domain (`bolehDiselesaikan(status, fotoBukti, hasilBukti)`), sehingga setiap pemanggil — HTTP hari ini, cron atau alat admin esok — memutuskan dengan cara yang sama.

Struktur berkas

```
src/
  index.ts          composition root: routes, fallback SPA, penanganan cron
  env.ts            binding infrastruktur dan konfigurasi (Env, AppEnv)
  domain/types.ts   enum, ReportRow/ReportDTO, urlFoto(), toDTO(), bolehDiselesaikan()
  adapters/
    llm.ts          prompt, few-shot, validasi keluaran, chatJSON(), periksaFotoBukti()
  storage.ts        R2: prefix, batas ukuran/tipe, simpan/ambil/baca/hapus foto
  session.ts        JWT dalam cookie, wajibPeran(), wajibPetugas, wajibAdmin
  password.ts       PBKDF2-SHA256 lewat WebCrypto, perbandingan waktu-konstan
  clock.ts          tanggalWIB(), rentangHariWIB()
  repositories/     reports, locations, users, activity, summaries
  services/
    report-service.ts kirim, ubah status (+ verifikasi bukti), hapus laporan
    analysis-service.ts analisis satu laporan, dijalankan setelah respons dikirim
    summary-service.ts ringkasan harian, statistik, data grafik, papan peringkat
    user-service.ts   daftar, masuk, kelola akun
    activity-service.ts tulis dan baca jejak audit
  routes/           auth, locations, reports, uploads, summary, activity, users, leaderboard

web/src/
  main.tsx          router: 8 rute halaman
  lib/i18n.tsx       kamus ID/EN, useBahasa(), useWaktuRelatif()
  lib/sesi.tsx       konteks sesi (useSesi)
  lib/api.ts         klien API bertipe, ApiError (status + data)
  components/Kop.tsx kepala halaman, tombol kembali, sakelar bahasa
  components/Lacak.tsx linimasa status tiga tahap
  components/Lencana.tsx lencana prioritas/status/kategori
  components/Panel*.tsx tab dashboard: Grafik, Aktivitas, Pengguna
  components/grafik/ warna tervalidasi, Garis, Batang, BatangBertumpuk, PetaPanas, Sparkline, KartuKPI
  pages/             Beranda, Lapor, StatusLaporan, LaporanPublik, Masuk, Daftar, Peringkat, Dashboard
web/public/peta-lokasi-gedung.jpg

migrations/         0001-0005, diterapkan berurutan
seed/toilets.sql     daftar toilet aktif
scripts/generate-qr.mjs QR per lantai + lembar cetak A4
scripts/build-frontend.mjs membangun frontend sebelum wrangler deploy
scripts/build-docs.mjs merender docs/*.html ke PDF
docs/               dokumentasi-teknis, panduan-umum (HTML + PDF)
qr-codes/           hasil generate-qr (13 PNG + cetak.html)
```

4. Model data

Basis data D1 (SQLite). Semua stempel waktu disimpan UTC dalam format `YYYY-MM-DD HH:MM:SS` ; konversi ke hari WIB dilakukan di `adapters/clock.ts` dan di SQL lewat `' +7 hours'` .

gedung

Kolom	Tipe	Keterangan
kode	TEXT PK	Satu huruf, A–J
nama	TEXT	Nama gedung dari poster “Peta Lokasi Gedung” (A Ki Hajar Dewantara, B Masjid At-Tarbiyah, C Dewi Sartika, D KH. Ahmad Dahlan, E Dr. Wahidin Sudirohusodo, F Daoed Joesoef, G RA. Kartini, H Mohamad Yamin, I Dr. Sutomo, J KH. Moh. Hasyim Ashari)
urutan	INTEGER	Urutan tampil di pemilih lokasi

toilets

Kolom	Tipe	Keterangan
id	TEXT PK	Format <code><gedung>--<lantai>--<JENIS></code> , mis. <code>A-2-PRIA</code> . Generator QR dan tampilan lokasi bergantung pada format ini
gedung_kode	TEXT FK → gedung	
lantai	INTEGER	
jenis	TEXT	<code>pria</code> <code>wanita</code> <code>disabilitas</code>
aktif	INTEGER	0/1. Toilet yang dihapus dari daftar dinonaktifkan, bukan dihapus, agar riwayat laporan tetap utuh
created_at	TEXT	

Indeks: `idx_toilets_lokasi` (`gedung_kode`, `lantai`, `jenis`) . Saat ini 26 toilet aktif di 13 lantai pada 6 gedung (A lantai 1–3, B 1, C 1–5, G 1, H 1, I 1–2); tiap lantai diasumsikan punya WC pria dan wanita.

toilet_info (view)

Menggabungkan `toilets` dan `gedung` , dan merakit nama tampilan di satu tempat: `'WC Pria – Lantai 1, Gedung A (Ki Hajar Dewantara)'` . Dashboard, ringkasan harian, dan konteks yang dikirim ke LLM semuanya memakai nama ini. Kolom: `id`, `gedung_kode`, `gedung_nama`, `gedung_urutan`, `lantai`, `jenis`, `aktif`, `nama` .

reports

Kolom	Tipe	Keterangan
id	TEXT PK	UUID v4
toilet_id	TEXT FK → toilets	
teks	TEXT	Keluhan mentah, 5–1000 karakter
foto_key	TEXT	Kunci objek R2 foto keadaan (<code>laporan/...</code>); wajib sejak versi berfoto
pelapor_id	TEXT FK → pengguna	NULL untuk laporan anonim; hanya yang terisi dihitung di papan peringkat
status	TEXT	<code>baru</code> → <code>diproses</code> → <code>selesai</code> (CHECK)
petugas	TEXT	Nama petugas yang terakhir mengubah status
selesai_at	TEXT	Diisi saat status menjadi selesai, dikosongkan bila dibuka kembali
foto_selesai_key	TEXT	Kunci objek R2 foto bukti (<code>bukti/...</code>); hanya foto yang lolos verifikasi
bukti_ai_hasil	TEXT	Putusan model vision atas foto bukti yang diterima; dalam praktik selalu <code>bersih</code> (foto <code>kotor</code> / <code>bukan_toilet</code> tidak pernah disimpan)

bukti_ai_alasan	TEXT	Satu kalimat dari model, maks. 300 karakter
bukti_ai_model	TEXT	Nama model vision saat itu
bukti_ai_ms	INTEGER	Latensi pemeriksaan foto
ai_status	TEXT	pending ok gagal (CHECK)
kategori	TEXT	Array JSON, mis. ["kebersihan", "perlengkapan"] ; dibaca dengan json_each di grafik
prioritas	TEXT	rendah sedang tinggi NULL (CHECK)
ringkasan	TEXT	Satu kalimat netral dari LLM, maks. 300 karakter
rekomendasi	TEXT	Instruksi kerja dari LLM, maks. 500 karakter
ai_error	TEXT	Pesan galat terakhir (maks. 500 karakter), untuk analisis ulang
ai_model	TEXT	Nama model teks saat itu
ai_ms	INTEGER	Latensi panggilan LLM teks — bahan kuantitatif bab hasil
created_at, updated_at	TEXT	

Indeks: created_at DESC ; (status, created_at DESC) ; (toilet_id, created_at DESC) ; ai_status WHERE ai_status <> 'ok' ; pelapor_id .

pengguna

Kolom	Tipe	Keterangan
id	TEXT PK	UUID; admin bawaan adm-0001
username	TEXT UNIQUE COLLATE NOCASE	3–30 karakter, [a-zA-Z0–9._-] ; ‘Budi’ dan ‘budi’ dianggap sama
nama	TEXT	2–60 karakter, ditampilkan di dashboard dan jejak audit
peran	TEXT	admin petugas pelapor (CHECK)
sandi_hash, sandi_salt	TEXT	PBKDF2-SHA256 100.000 iterasi, salt 16 byte per pengguna, keduanya heksadesimal
aktif	INTEGER	Akun nonaktif tidak bisa masuk; tidak pernah dihapus
created_at	TEXT	

aktivitas

Kolom	Tipe	Keterangan
id	INTEGER PK AUTOINCREMENT	
waktu	TEXT	UTC
aksi	TEXT	Sepuluh nilai, lihat bagian 10
report_id	TEXT	Tanpa foreign key: tetap tercatat walau laporannya sudah dihapus
pelaku	TEXT	Nama petugas/admin, atau pelapor / sistem
ringkas	TEXT	Satu baris yang langsung terbaca manusia
rincian	TEXT	JSON: perubahan nilai, putusan model, atau salinan utuh laporan yang dihapus

Indeks: waktu DESC ; report_id ; (aksi, waktu DESC) .

daily_summaries

Kolom	Tipe	Keterangan
tanggal	TEXT PK	YYYY-MM-DD menurut WIB

total_laporan	INTEGER	
ringkasan	TEXT	Paragraf 2–4 kalimat
sorotan	TEXT	Array JSON, maks. 4 poin tindakan
created_at	TEXT	Diperbarui setiap kali ringkasan dibuat ulang (<i>upsert</i>)

Migrasi

Berkas	Isi
0001_init.sql	Tabel <code>toilets</code> , <code>reports</code> , <code>daily_summaries</code> beserta indeks
0002_gedung_upi.sql	Tabel <code>gedung</code> berisi 10 gedung UPI Tasikmalaya; QR menjadi per lantai; view <code>toilet_info</code>
0003_bukti_dan_aktivitas.sql	Kolom <code>foto_selesai_key</code> ; tabel <code>aktivitas</code>
0004_pengguna.sql	Tabel <code>pengguna</code> menggantikan sandi bersama; kolom <code>pelapor_id</code> ; akun admin bawaan (<code>admin</code> / <code>Admin123!</code>)
0005_verifikasi_bukti.sql	Empat kolom <code>bukti_ai_*</code> untuk putusan model vision

Seed daftar toilet

`seed/toilets.sql` aman dijalankan berulang, termasuk setelah laporan ada: ia menonaktifkan semua toilet (`UPDATE toilets SET aktif = 0`) lalu mengaktifkan kembali yang tercantum lewat `INSERT ... ON CONFLICT(id) DO UPDATE` . Gedung D, E, F disiapkan sebagai baris berkomentar di bagian bawah; buka komentarnya begitu keberadaan toiletnya dipastikan, jalankan seed, lalu regenerasi QR — dalam urutan itu.

5. Lokasi dan kode QR

Satu kode QR mewakili **satu lantai** satu gedung, bukan satu toilet. URL yang dikodekan: `<BASE_URL>/lapor/<gedung>-<lantai>`, mis. `/lapor/A-1`. Jenis toilet (pria/wanita/disabilitas) dipilih pelapor di formulir. Keputusan ini memangkas jumlah stiker yang harus dicetak dan dirawat menjadi sepertiga.

- `GET /api/lokasi` mengelompokkan toilet aktif per gedung → daftar lantai; dipakai halaman depan (pilih manual) dan generator QR.
- `GET /api/lokasi/:id` memvalidasi pola `^[A-Za-z]-\d{1,2}$`, huruf dibesarkan, lalu mengembalikan toilet aktif pada lantai itu terurut pria → wanita → disabilitas. Kode yang tidak dikenal → 404, dan halaman formulir menawarkan pemilihan manual.

Generator QR

```
BASE_URL=https://ai-complaint.yvrtz.workers.dev npm run qr
```

Skrip mengambil `/api/lokasi` dari Worker (lokal `127.0.0.1:8787` bila `BASE_URL` kosong), **mengosongkan** folder `qr-codes/`, lalu menulis `<gedung>-<lantai>.png` (900 px) untuk tiap lantai dan `cetak.html` — lembar A4 berisi stiker dwibahasa 55 mm. Folder dikosongkan tiap kali agar kode dari daftar lokasi lama tidak pernah ikut tercetak. Saat ini: 13 kode (A-1, A-2, A-3, B-1, C-1...C-5, G-1, H-1, I-1, I-2).

6. Alur laporan dari ujung ke ujung

Pengiriman

1. Frontend memvalidasi: teks ≥ 5 karakter (maks. 1000), foto wajib, ukuran ≤ 5 MB.
2. `POST /api/uploads?jenis=laporan` (multipart, kolom `file`): tipe harus JPEG/PNG/WebP (415 bila bukan), ≤ 5 MB (413 bila lebih). Mengembalikan `{key, url}`.
3. `POST /api/reports` dengan `{toilet_id, teks, foto_key}`. Server memvalidasi ulang dengan Zod, memastikan toilet aktif (404 bila tidak), lalu memeriksa **kembar**: teks identik untuk toilet yang sama dalam 2 menit terakhir dianggap tombol kirim yang tertekan dua kali — id laporan lama dikembalikan dengan `duplikat: true` (200), tanpa analisis ulang.
4. Laporan baru disimpan (201), dicatat `aksi='lapor'`, dan analisis dijadwalkan lewat `ctx.waitUntil()`. Bila pengirim masuk sebagai pelapor, `pelapor_id` terisi.
5. Browser diarahkan ke `/laporan/<id>`.

Halaman status

Halaman memanggil `GET /api/reports/:id` setiap 2 detik selama `ai_status='pending'` (maksimal 20 kali ≈ 40 detik), lalu setiap 15 detik untuk menangkap perubahan status oleh petugas; tab yang tersembunyi diperiksa tiap 15 detik. Ditampilkan: linimasa tiga tahap (diterima → dikerjakan → selesai, dengan nama petugas), hasil analisis begitu tersedia, atau pesan bahwa analisis belum berhasil tetapi laporan tetap tercatat. Pelapor anonim diminta menyimpan tautan; pelapor berakun melihatnya juga di “Laporan saya” (`GET /api/reports/saya`, 50 terbaru).

Di dashboard

`GET /api/reports` mengurutkan: status (baru, diproses, selesai) → prioritas (tinggi, sedang, rendah, belum dianalisis) → terbaru. Filter: `status, prioritas, toilet_id, gedung, tanggal` (hari WIB), `limit` bawaan 100, maks. 200. Dashboard memuat ulang tiap 30 detik. Transisi status:

Tombol	Permintaan	Syarat
Kerjakan	<code>PATCH {status:'diproses'}</code>	—; <code>petugas</code> diisi nama pengubah
Tandai selesai	unggah foto <code>jenis=bukti</code> , lalu <code>PATCH {status:'selesai', foto_selesai_key}</code>	Foto harus lolos model vision (bagian 8). Tanpa foto → 400; ditolak → 422; pemeriksa gagal → 502
Analisis ulang	<code>POST /:id/analisa-ulang</code>	Muncul hanya bila <code>ai_status='gagal'</code> ; mengembalikan ke <code>pending</code> lalu menjalankan analisis; UI memuat ulang 3 detik kemudian
Hapus	<code>DELETE /:id</code>	Konfirmasi di browser; salinan utuh ditulis ke jejak audit, lalu baris dan kedua fotonya dihapus

7. Analisis LLM teks

Prompt berbasis aturan

Prompt sistem (bahasa Indonesia) memerankan model sebagai asisten petugas kebersihan dan mendefinisikan:

Kategori	Cakupan
kebersihan	kotor, jorok, noda, sampah berserakan, lantai dekil, coretan
perlengkapan	sabun/tisu/pengharum habis, tidak ada gayung, tempat sampah tidak tersedia
kerusakan	kloset mampet, keran/flush rusak, pintu/kunci/lampu rusak, kaca pecah
bau	bau tidak sedap, pesing, menyengat
genangan	lantai becek, air tergenang, meluap, bocor
lainnya	selain di atas; juga untuk teks yang tidak jelas atau bukan keluhan toilet

Prioritas	Kriteria (berbasis risiko keselamatan, bukan kata kunci)
tinggi	Ada risiko keselamatan atau toilet praktis tidak bisa dipakai: air meluap, kloset mampet, lantai licin parah, lampu mati total, pintu tidak bisa dikunci, bau sangat menyengat, pecahan kaca, banyak masalah sekaligus
sedang	Mengganggu kenyamanan tetapi toilet masih bisa dipakai: bau biasa, lantai kotor, sabun habis, tempat sampah penuh
rendah	Keluhan kecil atau antisipatif: tisu hampir habis, sedikit noda, cermin buram, saran

Aturan penulisan: `ringkasan` satu kalimat netral ≤ 15 kata dengan istilah baku; `rekomendasi` ≤ 2 kalimat, masalah yang paling berisiko mencelakakan diutamakan; dilarang mengarang masalah yang tidak disebut pelapor. Tiga contoh *few-shot* mengunci gaya dan penerapan aturan. Pesan pengguna berbentuk `Lokasi: ... \nKeluhan: ...`.

Validasi berlapis

```
respons JSON → Zod (kategori: string[] ≥1, prioritas, ringkasan, rekomendasi) → coerceAnalysis() → enum basis data
```

- Kategori dihuruf-kecilkan, dirapikan, yang di luar daftar dibuang; bila kosong → `["lainnya"]`.
- Prioritas tak dikenal → `sedang`.
- Ringkasan dipotong 300 karakter, rekomendasi 500.
- Bila model membungkus JSON dalam ````json`, objek pertama diekstrak dengan regex sebelum diparse.

Asinkron dan tahan gagal

`jalankanAnalisis()` dipanggil lewat `ctx.waitUntil()` setelah respons 201 dikirim. Ia tidak pernah melempar: sukses → `ai_status='ok'`, model dan latensi tersimpan, dicatat `aksi='analisis'`; gagal → `ai_status='gagal' + ai_error`, dicatat `aksi='analisis_gagal'`. Laporan tetap tampil di dashboard tanpa label dan dapat dianalisis ulang. Bila API DeepSeek mati, sistem menurun menjadi alat pelaporan biasa, bukan gagal.

Ringkasan harian

`ringkasHarian()` mengirim daftar bernomor `[prioritas] lokasi - ringkasan|teks` seluruh laporan pada satu hari WIB dan meminta paragraf 2–4 kalimat berangka konkret plus ≤ 4 sorotan tindakan ≤ 12 kata, tanpa mengarang di luar daftar. Hari tanpa laporan **tidak memanggil LLM**; disimpan kalimat tetap “Tidak ada keluhan yang masuk pada hari ini.”

8. Verifikasi foto bukti (LLM vision)

Foto bukti semula hanya membuktikan bahwa *sesuatu* difoto. Kini foto itu dinilai lebih dulu, di dalam `ubahStatus()` pada `report-service.ts`, sehingga berlaku untuk pemanggil mana pun.

```
petugas unggah foto → R2 (prefix bukti/)
PATCH /api/reports/:id {status:'selesai', foto_selesai_key}
├─ bacaFoto(): objek dibaca dari R2 (hanya prefix laporan/ dan bukti/),
├─ dikodekan base64 per potongan 32 KB (btoa), dikirim sebagai data URL
├─ periksaFotoBukti(): prompt sistem + [teks konteks, image_url detail:low]
├─ model menjawab {"toilet":bool,"bersih":bool,"keyakinan":0-1,"alasan":"..."}
├─ toilet=false → hasil 'bukan_toilet'
├─ toilet=true, bersih=false → hasil 'kotor'
├─ bersih → status=selesai, bukti_ai_* terisi, aksi='status' (200)
├─ kotor / bukan_toilet → foto dihapus dari R2, aksi='bukti_ditolak' (422)
├─ pemeriksa gagal → foto dihapus, aksi='verifikasi_gagal' (502)
```

Kriteria dalam prompt

- **toilet**: harus memperlihatkan bagian dalam toilet/kamar mandi (kloset, urinoir, wastafel, lantai kamar mandi, bilik). Koridor, orang, layar, langit-langit, foto gelap/blur → `false`.
- **bersih**: *tidak* bersih bila terlihat kotoran/noda, sampah, tisu bekas di lantai, genangan atau lantai basah merata, tempat sampah meluap, coretan, kerak tebal, atau bekas keluhan yang jelas belum ditangani. Noda permanen kecil, keramik tua, lantai lembap tipis bekas pel masih bersih.
- Keluhan asli (kategori + teks) dijadikan konteks: bila keluhannya terlihat di foto (sampah, genangan), periksa apakah sudah hilang; keluhan yang tak terlihat (bau, sabun habis) tidak boleh menjadi alasan menolak.
- Bila ragu, pilih tidak bersih. Dilarang mengarang detail. Alasan ≤ 25 kata, bahasa Indonesia.

Validasi keluaran

```
respons JSON → Zod (toilet: boolean longgar; keyakinan?: 0-1; alasan: string) → hasil ∈ {bersih, kotor, bukan_toilet}
```

Boolean yang datang sebagai string (“true”, “ya”, “yes”, “1”) diterima lewat `z.preprocess` — `z.coerce.boolean()` sengaja tidak dipakai karena akan mengubah string “false” menjadi `true`. Alasan dipotong 300 karakter.

Anggaran token untuk model “berpikir”

Gemini 3.x menghabiskan sebagian `max_tokens` untuk penalaran sebelum menjawab. Batas 300 token yang semula dipakai memotong JSON di tengah jalan dan muncul sebagai galat “Respons LLM bukan JSON”; anggaran dinaikkan ke 2.000. Jawaban akhirnya tetap pendek karena alasannya dibatasi 25 kata.

Apa yang disimpan, apa yang dibuang

- Foto yang **diterima**: kunci tersimpan di `foto_selesai_key`, putusan di `bukti_ai_hasil/alasan/model/ms`, dan salinan putusan ikut di `rincian` baris `aksi='status'`.
- Foto yang **ditolak** atau gagal diperiksa: dihapus dari R2 (bucket hanya berisi bukti yang lolos), putusan/galat tercatat di jejak audit beserta alasan, keyakinan, model, dan latensi — cukup untuk menghitung tingkat penolakan.
- Foto bukti dari sebelum fitur ini ada (`bukti_ai_hasil` NULL) tidak dianggap terverifikasi; bila laporan itu dibuka kembali dan hendak ditutup, petugas harus mengunggah foto baru.

Pemadaman layanan vision tidak menurunkan standar. Bila endpoint tidak dapat dihubungi, penutupan ditolak dengan 502 “coba lagi”, bukan diloloskan tanpa pemeriksaan. Ini berbeda dari analisis teks yang sengaja *degrade*: keluhan yang tidak dianalisis masih berguna, sedangkan bukti yang tidak diperiksa tidak.

9. Akun, sesi, dan wewenang

Peran	Cara akun dibuat	Wewenang
admin	Bawaan migrasi 0004 (<code>admin / Admin123!</code>)	Semua wewenang petugas + kelola akun petugas (<code>/api/pengguna</code>)
petugas	Dibuat admin	Daftar dashboard, ubah status, unggah bukti, analisis ulang, hapus laporan, statistik/grafik, jejak audit, buat ringkasan
pelapor	Mendaftar sendiri di <code>/daftar</code>	Melapor atas nama, “Laporan saya”, posisi di papan peringkat
tanpa akun	—	Melapor anonim, melacak lewat tautan, membaca papan publik dan peringkat

Aspek	Penerapan
Kata sandi	PBKDF2-SHA256, 100.000 iterasi, salt 16 byte per pengguna, WebCrypto; teks asli tidak pernah disimpan. Mengganti sandi juga mengganti salt
Sesi	JWT HS256 (<code>{id, nama, peran, exp}</code>) ditandatangani <code>AUTH_SECRET</code> , dalam cookie <code>sesi : HttpOnly , SameSite=Lax , Secure</code> di HTTPS, <code>path=/</code> , umur 12 jam (satu shift)
Wewenang	<code>wajibPeran(...peran)</code> : 401 tanpa sesi, 403 bila peran tidak termasuk. Admin tidak mendapat akses menyeluruh secara implisit — setiap endpoint menyebut sendiri peran yang boleh (<code>wajibPetugas = admin+petugas</code> , <code>wajibAdmin</code>)
Pendaftaran	Hanya pernah membuat <code>pelapor</code> ; akun petugas hanya dari admin. Username unik tanpa peduli huruf besar-kecil (409 bila dipakai)
Enumerasi akun	Akun tak dikenal, akun nonaktif, dan sandi salah semuanya dijawab 401 dengan pesan yang sama
Perbandingan hash	Waktu-konstan (XOR seluruh karakter), agar lama respons tidak membocorkan isi hash
Pengaman admin	Admin tidak dapat menonaktifkan akunnya sendiri (400 “kunci diri sendiri”)
Masuk petugas dicatat	Sign-in admin/petugas ditulis ke jejak audit (<code>aksi= 'masuk'</code>); sign-in pelapor tidak

10. Jejak audit

Setiap pergerakan ditulis ke `aktivitas` lewat `catat()` . Kegagalan mencatat **tidak pernah** menggagalkan tindakan yang dicatat — menerima laporan lebih penting daripada merekamnya — galatnya hanya dicetak ke log Worker. Baris tidak pernah diubah atau dihapus lewat aplikasi.

aksi	Pelaku	Kapan	Isi rincian
lapor	nama pelapor / <code>pelapor</code>	Laporan baru diterima	<code>toilet_id</code> , teks
analisis	sistem	Analisis teks selesai	kategori, prioritas, model
analisis_gagal	sistem	Analisis teks gagal	error (≤300 karakter)
status	petugas	Status berubah	dari, ke, foto_bukti, verifikasi {hasil, alasan, model, ms}
bukti_ditolak	petugas	Foto bukti dinilai kotor/bukan toilet	hasil, alasan, keyakinan, model, ms
verifikasi_gagal	petugas	Layanan vision tidak dapat dihubungi / respons rusak	error (≤300 karakter), model
hapus	petugas	Laporan dihapus	salinan: <code>toilet_nama</code> , teks, status, prioritas, kategori, ringkasan, dibuat, <code>ada_foto</code>
masuk	admin/petugas	Sign-in ke dashboard	—
ringkasan	sistem	Ringkasan harian disusun (cron atau tombol)	—

pengguna	admin	Akun petugas dibuat/diubah	—; ringkas menyebut kolom yang diubah
----------	-------	----------------------------	---------------------------------------

Tab Aktivitas di dashboard memfilter per aksi, menampilkan 100 baris terbaru (maks. 300 lewat API), dan membentangkan salinan laporan yang dihapus secara langsung tanpa perlu diklik.

11. Ringkasan harian, statistik, dan grafik

Cron

"crons": ["0 10 * * *"] — 10.00 UTC = 17.00 WIB. Penangan `scheduled()` memanggil `buatRingkasanHarian(tanggalWIB())` lewat `ctx.waitUntil()`. Tombol “Buat ulang” di dashboard (`POST /api/summary/generate`) menjalankan hal yang sama kapan saja dan menimpa ringkasan hari itu (*upsert*). Secara lokal, cron dapat dipicu dengan `wrangler dev --test-scheduled` lalu membuka `/__scheduled`.

Kartu angka (`GET /api/summary/stats`)

Untuk satu hari WIB (bawaan hari ini): total laporan, jumlah per prioritas, jumlah analisis gagal; **belum selesai** (sepanjang waktu, status $\neq$ selesai); lima lokasi terbanyak hari itu.

Data grafik (`GET /api/summary/grafik?hari=14` , maks. 90)

Dihitung dalam dua *batch* D1. Hari kosong diisi nol, bukan dilewati — celah terbaca sebagai data hilang, nol terbaca sebagai “tidak ada laporan”, dan itulah kebenarannya.

Bidang	Isi	Rentang	Pertanyaan yang dijawab di tab Grafik
tren	laporan, laporan_lalu, perubahan %, selesai, tinggi	jendela n hari vs n hari sebelumnya	Empat kartu KPI: laporan masuk (+/- %), prioritas tinggi, tingkat penyelesaian, rata-rata waktu penyelesaian
harian	total, selesai per tanggal	n hari	Grafik garis “Laporan 14 hari terakhir”
harianPrioritas	tinggi/sedang/rendah per tanggal	n hari	Batang bertumpuk: apakah beban yang berat bertambah, atau hanya jumlahnya?
jamHari	jumlah per hari-dalam-minggu $\times$ jam WIB	sepanjang waktu	Peta panas “Kapan keluhan masuk” — dasar jadwal ronda
matriks	jumlah per gedung $\times$ kategori	sepanjang waktu	Peta panas “Gedung $\times$ jenis masalah”
waktuPrioritas	jumlah dan rata-rata menit penyelesaian per prioritas	sepanjang waktu, yang sudah selesai	“Waktu penyelesaian per prioritas” — uji nyata: prioritas tinggi harus lebih cepat
gedung	jumlah per gedung, 10 teratas	sepanjang waktu	Batang “Laporan per gedung”
kategori, prioritas	jumlah per kategori / per prioritas	sepanjang waktu	Sebaran (kategori dihitung dengan <code>json_each</code> , satu laporan dapat masuk beberapa kategori)
penyelesaian	jumlah selesai dan rata-rata menit	sepanjang waktu	Kartu “Rata-rata waktu penyelesaian”

Waktu penyelesaian = `selesai_at - created_at` dalam menit (`julianday`). Pengelompokan per hari memakai `date(created_at, '+7 hours')` sehingga “hari” berarti hari WIB.

12. Papan publik dan papan peringatan

Papan publik (`/laporan`)

Dapat dibaca tanpa login. `GET /api/reports/publik` memakai filter yang sama dengan dashboard tetapi **hanya** mengembalikan: id, status, prioritas, kategori, ringkasan, ai_status, waktu, selesai_at, lokasi, dan URL foto bukti. Teks mentah, foto pelapor, dan nama petugas sengaja tidak ikut — agar papan tidak menjadi saluran teks keluhan yang belum disaring atau memuat wajah yang tak sengaja terfoto. Foto bukti ditampilkan karena itulah yang membuat klaim “sudah ditangani” dapat diperiksa siapa pun. Ikut dikembalikan hitungan `{total, selesai}` seluruh laporan.

Papan peringatan (`/peringkat`)

20 pelapor teratas diurutkan jumlah laporan, lalu jumlah selesai, lalu nama. Hanya laporan dengan `pelapor_id` yang dihitung. Pelapor yang sedang masuk juga menerima posisinya sendiri (`saya: {peringkat, laporan}` ; `peringkat = 1 + jumlah pelapor yang laporannya lebih banyak`), termasuk bila berada di luar 20 besar.

13. Penyimpanan foto

Aspek	Penerapan
Bucket	R2 kato , binding BUCKET , tanpa akses publik
Kunci objek	<laporan bukti>/<YYYY-MM-DD>/<uuid>.<jpg png webp> ; prefix tanggal memudahkan penelusuran dan pembersihan per periode
Batas unggah	≤ 5 MB; tipe image/jpeg , image/png , image/webp ; tanpa kompresi di sisi klien
Unggah	Lewat binding (Worker mengalirkan body ke bucket), bukan presigned URL — tidak ada kredensial S3 di mana pun
Penyajian	GET /api/uploads/<key> hanya untuk prefix laporan/ dan bukti/ ; Cache-Control: public, max-age=31536000, immutable (nama UUID tak pernah ditimpa); ETag + If-None-Match → 304
Pembacaan untuk vision	bacaFoto() memuat seluruh objek ke memori lalu dikodekan base64 — 5 MB → ~6,7 MB data URL, jauh di bawah batas memori Worker
Penghapusan	Saat laporan dihapus: kedua foto dihapus setelah baris dan log ditulis; foto bukti yang ditolak/gagal diperiksa dihapus seketika. Penghapusan foto tidak pernah menggagalkan operasi induknya
Mengapa bukan r2.dev	DNS pub-*.r2.dev dibajak sebagian ISP Indonesia — terverifikasi saat pengembangan: resolver lokal mengembalikan alamat non-Cloudflare dan TLS gagal, sedangkan lewat 1.1.1.1 benar. Menyajikan dari origin sendiri menghilangkan ketergantungan itu

14. Referensi API lengkap

Semua respons JSON. Galat selalu berbentuk {"error": "pesan"} , kadang dengan bidang tambahan. Body permintaan JSON kecuali unggah (multipart). Sesi dibawa oleh cookie.

Publik

Endpoint	Permintaan	Respons	Galat
GET /api/health	—	{ok:true, waktu}	
POST /api/auth/masuk	{username, password}	{id, nama, peran}	400 kosong; 401 salah/nonaktif
POST /api/auth/daftar	{username, nama, password}	201 {id, nama, peran:'pelapor'}	400 validasi; 409 username dipakai
POST /api/auth/keluar	—	{ok:true}	
GET /api/auth/saya	—	{id, nama, peran}	401 belum masuk
GET /api/lokasi	—	{data:[{kode, nama, lantai:[...]}]}	
GET /api/lokasi:id	A-1	{gedung_kode, gedung_nama, lantai, toilets:[{id, jenis}]}	404 tidak dikenal
POST /api/uploads?jenis=laporan bukti	multipart file	201 {key, url}	400 tanpa berkas; 413 > 5 MB; 415 tipe
GET /api/uploads/:key	—	biner + cache header; 304	404 prefix salah/tak ada
POST /api/reports	{toilet_id, teks, foto_key}	201 {id, toilet, duplikat:false} atau 200 {..., duplikat:true}	400 validasi; 404 toilet tak dikenal
GET /api/reports/publik	?status&prioritas&limit	{data:[LaporanPublik], jumlah:{total, selesai}}	
GET /api/reports/saya	—	{data:[Laporan]} (kosong bila bukan pelapor)	
GET /api/reports/:id	—	Laporan (DTO lengkap)	404
GET /api/peringkat	—	{data:[{id, nama, laporan, selesai}], saya:{peringkat, laporan null}}	

Petugas dan admin (wajibPetugas)

Endpoint	Permintaan	Respons	Galat
GET /api/reports	? status&prioritas&toilet_id&gedung&tanggal&limit	{data: [Laporan]}	401/403
PATCH /api/reports/:id	{status, foto_selesai_key?}	{ok:true, status, verifikasi: {hasil, alasan, model, ms} null}	400 status tak valid / foto tak ditemukan / bukti kurang; 404; 422 {error, hasil:'kotor' 'bukan_toilet', alasan} ; 502 pemeriksaan gagal
POST /api/reports/:id/analisa- ulang	—	{ok:true}	404
DELETE /api/reports/:id	—	{ok:true}	404
GET /api/summary?tanggal=	bawaan hari ini WIB	{ada:false, tanggal} atau {ada:true, tanggal, total_laporan, ringkasan, sorotan}	400 format tanggal
GET /api/summary/stats? tanggal=		{tanggal, hari_ini: {total, tinggi, sedang, rendah, ai_gagal}, belum_selesai, lokasi_teratas: [{lokasi, jumlah}]}	400
GET /api/summary/grafik? hari=14	maks. 90	lihat bagian 11	
POST /api/summary/generate? tanggal=		{ada:true, tanggal, total_laporan, ringkasan, sorotan}	400; 502 LLM gagal
GET /api/aktivitas	?aksi&report_id&limit (maks. 300)	{data:[{id, waktu, aksi, report_id, pelaku, ringkas, rincian}]}	

Admin (wajibAdmin)

Endpoint	Permintaan	Respons	Galat
GET /api/pengguna	—	{data:[{id, username, nama, peran, aktif, created_at}]} (admin + petugas)	401/403
POST /api/pengguna	{username, nama, password}	201 akun baru berperan petugas	400; 409
PATCH /api/pengguna/:id	{nama?, password?, aktif?} minimal satu	{ok:true}	400 kosong / kunci diri sendiri; 404

Kode status yang dipakai

200 sukses · 201 dibuat · 304 belum berubah · 400 permintaan tak valid atau aturan bisnis sederhana · 401 belum masuk · 403 peran tidak
berhak · 404 tidak ditemukan · 409 konflik username · 413 berkas terlalu besar · 415 tipe berkas · **422 foto bukti ditolak model** · 500 galat tak
tertangani (`app.onError`) · 502 layanan LLM/vision gagal. Path `/api/*` yang tak dikenal → 404 JSON; path lain yang bukan aset →
`index.html` (fallback SPA).

15. Antarmuka pengguna

Rute	Halaman	Isi
/	Beranda	Peta gedung (dapat diperbesar), pilih gedung → lantai manual (cadangan bila QR rusak), “Laporan saya” bagi pelapor yang masuk, tautan papan publik, peringkat, dan masuk petugas
/lapor/:lokasiId	Lapor	Tujuan QR. Pilih jenis toilet, teks (contoh cepat tersedia), foto wajib dengan <code>capture="environment"</code> , kirim. Menampilkan identitas pelapor atau catatan anonim
/laporan/:id	StatusLaporan	Konfirmasi, linimasa tiga tahap, hasil analisis (polling), saran menyimpan tautan
/laporan	LaporanPublik	Papan terbuka dengan filter status/prioritas dan hitungan “x dari y selesai”
/masuk	Masuk	Satu formulir untuk semua peran; pelapor diarahkan ke beranda, petugas/admin ke dashboard
/daftar	Daftar	Pendaftaran pelapor
/peringkat	Peringkat	20 besar dan posisi sendiri
/petugas	Dashboard	Tab Laporan / Grafik / Aktivitas / Akun Petugas (admin). Pelapor dan tamu diarahkan ke <code>/masuk</code> ; server tetap menolak walau tab dipaksa tampil

Kartu laporan di dashboard

Tepi kiri berwarna menandai prioritas (merah tinggi, kuning sedang, hijau rendah) agar terbaca dari seberang ruangan. Isi: rencana prioritas/status/kategori, waktu relatif, lokasi, ringkasan (atau teks mentah bila belum dianalisis), teks asli, kotak “Tindakan”, foto keadaan, foto bukti berbingkai hijau dengan keterangan “✓ Bukti penyelesaian · Diverifikasi AI: bersih” dan alasan model. Tombol “Tandai selesai” berubah menjadi “Mengunggah...” lalu “AI memeriksa foto...”; putusan penolakan tampil sebagai kotak merah “Foto ditolak” beserta “Penilaian AI: ...” sampai percobaan berikutnya.

Bahasa, warna, responsivitas

- Sakelar ID/EN di kepala halaman; pilihan disimpan di `localStorage` (`bahasa`). Keluaran LLM tetap Indonesia karena pembacanya petugas.
- Palet maroon, bata, krem diambil dari poster “Peta Lokasi Gedung” UPI Tasikmalaya.
- Palet grafik (`grafik/warna.ts`) lolos pemeriksaan rentang terang, ambang chroma, keterpisahan buta warna (deutan/protan/tritan), dan kontras; ramp sekuensial peta panas monoton dalam luminansi (0,933 → 0,164). Jangan diubah tanpa menjalankan ulang validatornya.
- Seluruh halaman dirancang untuk layar ponsel lebih dulu — formulir laporan dibuka dari kamera QR.

16. Konfigurasi, variabel, dan rahasia

Nama	Jenis	Nilai / keterangan
LLM_BASE_URL	var	<code>https://api.deepseek.com</code>
LLM_MODEL	var	<code>deepseek-chat</code>
VISION_BASE_URL	var	<code>https://generativelanguage.googleapis.com/v1beta/openai</code>
VISION_MODEL	var	<code>gemini-3.6-flash</code> ; alternatif <code>gemini-3.5-flash-lite</code> (kuota gratis lebih besar)
LLM_API_KEY	secret	Kunci DeepSeek
VISION_API_KEY	secret	Kunci Gemini dari <code>aistudio.google.com/apikey</code> . Bila kosong, jatuh ke <code>LLM_API_KEY</code> — hanya berguna bila kedua model di penyedia yang sama
AUTH_SECRET	secret	String acak panjang untuk menandatangani JWT; menggantinya membatalkan semua sesi
DB, BUCKET, ASSETS	binding	D1 <code>kato</code> , R2 <code>kato</code> , aset statis <code>./dist</code>

`Vars` ada di `wrangler.jsonc` dan ter-commit; `secrets` dipasang dengan `npx wrangler secret put` dan tidak pernah ada di repositori. Untuk lokal, salin `.dev.vars.example` menjadi `.dev.vars` (diabaikan git) dan isi ketiga rahasia. Mengganti penyedia vision: ubah dua `vars` dan satu `secret`; format panggilan (OpenAI chat completions dengan `image_url`) didukung OpenAI (`https://api.openai.com/v1`, `gpt-4o-mini`), OpenRouter (`https://openrouter.ai/api/v1`), dan Anthropic (`https://api.anthropic.com/v1`, `claude-haiku-4-5-20251001`).

17. Pengembangan lokal

```
npm install
cp .dev.vars.example .dev.vars # isi LLM_API_KEY, VISION_API_KEY, AUTH_SECRET
npm run db:local # migrasi ke D1 lokal
npm run db:seed # daftar toilet
npm run build # frontend harus dibangun sekali sebelum wrangler dev
npm run dev # Vite (5173) + Worker (8787) berdampingan
```

Skrip	Fungsi
dev / dev:api / dev:web	Keduanya bersamaan (<code>concurrently</code>) / Worker saja / Vite saja
build	Vite build ke <code>./dist</code>
typecheck	<code>tsc --noEmit</code> untuk backend, frontend, dan skrip sekaligus
db:local / db:remote	Menerapkan migrasi yang belum dijalankan
db:seed / db:seed:remote	Menjalankan <code>seed/toilets.sql</code>
qr	Menghasilkan QR + lembar cetak (butuh <code>BASE_URL</code>)
docs	Merender kedua dokumen ke PDF lewat Chrome headless
cf-typegen	Menghasilkan tipe binding dari wrangler
deploy	<code>wrangler deploy</code> (membangun frontend lebih dulu lewat <code>build.command</code>)

Menguji tanpa kunci API sungguhan

Karena kedua model dipanggil lewat URL yang dapat dikonfigurasi, verifikasi foto dapat diuji dengan server tiruan lokal yang menjawab format chat completions, lalu Worker dijalankan dengan `wrangler dev --var VISION_BASE_URL:http://127.0.0.1:8799/v1` . Skenario yang diverifikasi dengan cara ini sebelum rilis: selesai tanpa foto (400), foto bersih (200, kolom terisi), foto kotor (422, foto hilang dari R2 → 404), bukan toilet (422), kunci foto salah (400), diproses tanpa foto (200), layanan mati (502, tercatat `verifikasi_gagal`).

18. Penyebaran dan operasi

```
npx wrangler secret put LLM_API_KEY # DeepSeek
npx wrangler secret put VISION_API_KEY # Gemini
npx wrangler secret put AUTH_SECRET

npm run db:remote # migrasi D1 produksi – jalankan SEBELUM deploy
npm run db:seed:remote # daftar toilet
npm run deploy
```

- **Urutan penting:** kode baru menulis kolom `bukti_ai_*` ; bila migrasi 0005 belum diterapkan, penutupan laporan gagal dengan galat SQL.
- `wrangler.jsonc` : `run_worker_first` : `["/api/*", "/__scheduled"]` sehingga API selalu mencapai Worker; path lain yang tidak cocok aset dikembalikan sebagai `index.html` (`not_found_handling` : `single-page-application`). `observability` aktif.
- **Workers Builds** (deploy otomatis dari GitHub): perintah `deploy` `npx wrangler deploy` , perintah build dikosongkan; frontend dibangun oleh `build.command` yang juga memasang dependensi pada checkout bersih. Rahasia dan migrasi tidak otomatis. Bila push tidak memicu deploy, periksa Workers & Pages → ai-complaint → Settings → Builds: repo tersambung, branch `main` , dan GitHub App Cloudflare punya akses ke repo.
- Periksa keadaan produksi dari terminal: `npx wrangler deployments list` , `npx wrangler secret list` , `npx wrangler d1 migrations list` kato `--remote` , `curl https://ai-complaint.yvrtz.workers.dev/api/health` .
- Log langsung: `npx wrangler tail` .

Wajib dilakukan sebelum dipakai sungguhan: ganti kata sandi admin bawaan (tab Akun Petugas, atau `PATCH /api/pengguna/adm-0001`). Nilainya tercatat di berkas migrasi yang ter-commit sehingga harus dianggap publik.

19. Pemantauan dan pemecahan masalah

Menghitung pemakaian model vision dari aplikasi sendiri

```
npx wrangler d1 execute kato --remote --command "
SELECT date(waktu) AS hari, aksi, COUNT(*) AS jumlah
FROM aktivitas
WHERE aksi IN ('bukti_ditolak','verifikasi_gagal')
OR (aksi='status' AND rincian LIKE '%"hasil\":"\\"bersih\%"')
GROUP BY hari, aksi ORDER BY hari DESC"
```

Jumlah ketiga baris per hari = jumlah panggilan ke Gemini hari itu. Angka batas dan pemakaian resmi ada di `aistudio.google.com/rate-limits` (per project) dan Google Cloud Console → APIs → Generative Language API → Metrics/Quotas.

Membaca galat verifikasi terakhir

```
npx wrangler d1 execute kato --remote --command "
SELECT waktu, rincian FROM aktivitas WHERE aksi='verifikasi_gagal' ORDER BY id DESC LIMIT 3"
```

Latensi dan tingkat keberhasilan (bahan bab hasil)

```
-- analisis teks
SELECT ai_status, COUNT(*) n, ROUND(AVG(ai_ms)) rata_ms, MIN(ai_ms) min_ms, MAX(ai_ms) maks_ms
FROM reports GROUP BY ai_status;
-- verifikasi foto yang diterima
SELECT COUNT(*) n, ROUND(AVG(bukti_ai_ms)) rata_ms FROM reports WHERE bukti_ai_hasil='bersih';
-- sebaran prioritas dan kategori
SELECT prioritas, COUNT(*) FROM reports GROUP BY prioritas;
SELECT je.value kategori, COUNT(*) FROM reports, json_each(kategori) je GROUP BY je.value;
```

Gejala → penyebab → tindakan

Gejala	Penyebab yang lazim	Tindakan
“Pemeriksaan foto gagal. Coba lagi sebentar.” terus-menerus	Lihat <code>verifikasi_gagal</code> : LLM HTTP 404 ... no longer available = nama model kedaluwarsa; 401/403 = kunci salah/kosong; 429 = kuota habis; “Respons LLM bukan JSON” = JSON terpotong (anggaran token) atau model tidak patuh	Ganti <code>VISION_MODEL</code> ke nama yang disebut galat; pasang ulang <code>VISION_API_KEY</code> ; pindah ke model lite atau aktifkan billing; periksa <code>max_tokens</code>
Foto bersih ditolak berulang	Foto gelap/blur, sudut sempit, atau lantai masih basah merata	Foto ulang dengan cahaya cukup, tampak kloset + lantai; keringkan lantai. Alasan model tampil di kartu — gunakan untuk mengoreksi
Kartu laporan tanpa label AI, “Analisis gagal” > 0	DeepSeek tidak terjangkau / kunci salah; lihat <code>ai_error</code>	Perbaiki kunci, tekan “Analisis ulang”
Push ke GitHub tidak men-deploy	Workers Builds belum tersambung / branch salah / build gagal	Dashboard Cloudflare → Builds; sementara itu <code>npm run deploy</code>
Galat SQL <code>no such column: bukti_ai_hasil</code>	Deploy sebelum migrasi	<code>npm run db:remote</code>
Semua pengguna terlempar keluar	<code>AUTH_SECRET</code> diganti	Wajar; masuk kembali
QR mengarah ke “Kode lokasi tidak dikenal”	Toilet lantai itu dinonaktifkan di seed, atau QR dari daftar lama	Perbarui seed → <code>db:seed:remote</code> → <code>npm run qr</code> → cetak ulang
Foto tidak tampil di jaringan kampus	Seharusnya tidak terjadi (disajikan dari origin sendiri); bila terjadi, periksa prefix kunci	Hanya <code>laporan/</code> dan <code>bukti/</code> yang disajikan
Ringkasan harian kosong pukul 17.05	Cron gagal (LLM) atau belum ada laporan	Lihat <code>aksi='ringkasan'</code> ; tekan “Buat ulang”

20. Keputusan rancangan yang berbiaya

Masing-masing adalah keputusan yang mengorbankan sesuatu di tempat lain, sehingga alasannya dicatat.

Analisis berjalan setelah respons dikirim

Pelapor menerima konfirmasi seketika sementara panggilan LLM berlanjut di latar; halaman konfirmasi melakukan polling. Panggilan sinkron akan menaruh jeda tiga detik antara mahasiswa dan tombol kirim.

Panggilan LLM yang gagal tidak pernah menghilangkan laporan

Baris ditandai `gagal` beserta teks galatnya, tetap tampil, dan dapat dianalisis ulang. Sistem menurun menjadi alat pelaporan biasa, bukan gagal.

Keluaran LLM tidak pernah dipercaya apa adanya

Setiap respons divalidasi Zod lalu dipaksa ke enum yang dikenal basis data — untuk teks maupun vision.

Penyelesaian menuntut bukti yang diperiksa, ditolak di server

Foto keadaan dan foto bukti divalidasi di server, bukan sekadar disembunyikan di antarmuka. Selesai tanpa foto → 400; dengan foto yang dinilai kotor/bukan toilet → 422 beserta alasan. Biayanya: petugas menunggu beberapa detik dan sistem bergantung pada penyedia model kedua.

Pemadaman vision tidak diloloskan

Lihat bagian 8. Kebalikan dari analisis teks, dan disengaja.

Jejak audit sengaja tanpa foreign key

Laporan yang *dihapus*-lah yang paling perlu terlacak. Foreign key akan memaksa baris audit ikut terhapus — kebalikan dari tujuannya. Seluruh isi laporan disalin ke `rincian` sebelum barisnya hilang.

Melapor tidak butuh akun, tetapi akun dihargai

Dinding pendaftaran di pintu toilet mematikan adopsi. Laporan berakun masuk papan peringkat; laporan anonim tetap diterima dan ditangani, hanya tidak diperingkat. Laporan menempel pada akun, bukan perangkat — versi awal yang memakai `localStorage` kehilangan daftar saat data situs dibersihkan.

Satu QR per lantai

Sepertiga jumlah stiker, dengan biaya satu ketukan tambahan (pilih jenis toilet) di formulir.

Foto tidak melewati domain pihak ketiga

Lihat bagian 13.

Antarmuka dwibahasa, analisis tetap Indonesia

Label mengikuti pembaca; ringkasan dan rekomendasi selalu Indonesia karena yang bertindak adalah petugas kebersihan. Keluhan berbahasa Inggris tetap dipahami dan diringkas dalam bahasa Indonesia.

Warna grafik dihitung, bukan dikira-kira

Lihat bagian 15.

21. Batasan yang diketahui dan pekerjaan lanjutan

Batasan	Dampak	Arah perbaikan
Tidak ada pembatasan laju atau CAPTCHA pada <code>POST /api/reports</code>	Spam bisa masuk; hanya deteksi kembar 2 menit yang menahannya. Petugas dapat menghapus (tercatat)	Rate limiting per IP lewat Cloudflare, atau Turnstile
Tidak ada notifikasi ke petugas	Dashboard harus terbuka (memuat ulang tiap 30 detik)	Web Push atau WhatsApp/Telegram untuk prioritas tinggi
Verifikasi foto dapat dikelabui	Foto toilet lain atau foto lama yang bersih akan lolos; tidak ada pemeriksaan EXIF/waktu/lokasi	Wajibkan kamera langsung (bukan galeri), bandingkan dengan foto keadaan, atau minta dua sudut
Putusan model bersifat probabilistik	Ada kemungkinan salah tolak/salah terima; belum ada ukuran akurasi terhadap penilaian manusia	Uji dengan set foto berlabel; laporkan presisi/recall di bab hasil
Tingkat gratis Gemini	Kuota harian; data dapat dipakai Google	Model lite / billing (lihat bagian 16)

Nama model penyedia berumur pendek	404 mendadak saat model ditutup	Pantau <code>verifikasi_gagal</code> ; pertimbangkan alias "latest" bila penyedia menyediakannya
Tidak ada alur lupa sandi	Petugas yang lupa harus minta admin mengatur sandi baru; admin yang lupa harus diperbaiki lewat SQL	Reset lewat e-mail/OTP
Belum ada uji otomatis	Verifikasi dilakukan manual (curl + server tiruan)	Vitest + <code>unstable_dev</code> wrangler
Gedung D, E, F belum terdaftar; jenis toilet per lantai diasumsikan	Lokasi tersebut tidak dapat dilaporkan	Pastikan di lapangan, perbarui seed, regenerasi QR
Foto tidak dikompresi di klien	Unggahan hingga 5 MB di jaringan seluler	Kompresi <code>canvas</code> sebelum unggah
Keluhan berbahasa selain Indonesia/Inggris	Belum diuji	—

22. Lampiran

A. Contoh respons

POST /api/reports → 201

```
{"id":"8f0c...", "toilet":"WC Pria – Lantai 1, Gedung A (Ki Hajar Dewantara)", "duplikat":false}
```

GET /api/reports/:id setelah analisis

```
{"id":"8f0c...", "toilet_id":"A-1-PRIA", "toilet_nama":"WC Pria – Lantai 1, Gedung A (Ki Hajar Dewantara)",
  "gedung_kode":"A", "gedung_nama":"Ki Hajar Dewantara", "lantai":1, "jenis":"pria",
  "teks":"WC lantai 2 bau banget, lantainya becek, sama sabunya habis.",
  "foto_url":"/api/uploads/laporan/2026-09-11/...jpg", "foto_selesai_url":null,
  "bukti_ai_hasil":null, "bukti_ai_alasan":null, "bukti_ai_model":null, "bukti_ai_ms":null,
  "status":"baru", "petugas":null, "selesai_at":null,
  "ai_status":"ok", "kategori":["bau", "genangan", "perlengkapan"], "prioritas":"tinggi",
  "ringkasan":"Toilet berbau menyengat, lantai tergenang air, dan sabun habis.",
  "rekomendasi":"Keringkan lantai lebih dulu karena berisiko membuat pengguna terpeleset. Setelah itu bersihkan sumber bau dan isi ulang sabun.",
  "ai_error":null, "ai_model":"deepseek-chat", "ai_ms":1130, "pelapor_id":null,
  "created_at":"2026-09-11 08:40:12", "updated_at":"2026-09-11 08:40:14"}
```

PATCH /api/reports/:id → 200 (foto diterima)

```
{"ok":true, "status":"selesai",
  "verifikasi":{"hasil":"bersih", "alasan":"Kloset dan lantai terlihat bersih tanpa sampah.", "model":"gemini-3.6-flash", "ms":4210}}
```

PATCH /api/reports/:id → 422 (foto ditolak)

```
{"error":"Toilet pada foto masih terlihat kotor. Bersihkan lagi, lalu foto ulang.",
  "hasil":"kotor", "alasan":"Lantai masih basah dan ada tisu bekas."}
```

Keluaran mentah model vision yang diharapkan

```
{"toilet":true, "bersih":false, "keyakinan":0.9, "alasan":"Lantai masih basah dan ada tisu bekas."}
```

B. Glosarium

Istilah	Arti dalam dokumen ini
LLM	<i>Large language model</i> ; model bahasa terlatih yang dipakai apa adanya lewat API
Model vision	LLM yang menerima gambar sebagai masukan (<i>multimodal</i>)
Few-shot	Contoh pasangan masukan–keluaran di dalam prompt untuk mengunci gaya jawaban
JSON mode / <code>response_format</code>	Permintaan agar model menjawab dengan JSON valid saja
Model “berpikir”	Model yang menghasilkan token penalaran internal sebelum jawaban; ikut menghabiskan <code>max_tokens</code>
Worker	Fungsi tanpa server di jaringan tepi Cloudflare yang menjalankan seluruh backend dan menyajikan frontend
D1 / R2	Basis data SQLite terkelola / penyimpanan objek Cloudflare
Binding	Cara Worker mengakses D1/R2 tanpa kredensial jaringan
Migrasi	Berkas SQL ber-nomor yang mengubah skema secara berurutan
Seed	Data awal (daftar toilet) yang aman dijalankan ulang
JWT	Token sesi bertanda tangan; disimpan di cookie HttpOnly
PBKDF2	Fungsi turunan kunci untuk menyimpan sandi secara aman
<code>waitUntil</code>	Mekanisme Workers untuk melanjutkan pekerjaan setelah respons dikirim

Cron	Pemicu terjadwal Cloudflare; di sini 17.00 WIB setiap hari
WIB	Waktu Indonesia Barat, UTC+7; semua “hari” dalam aplikasi mengikuti WIB
Jejak audit	Tabel <code>aktivitas</code> ; catatan yang tidak dapat dihapus dari aplikasi
Bukti	Foto dari petugas yang menunjukkan kondisi setelah ditangani

Repositori: github.com/rdnsa/AI-complaint · Aplikasi: ai-complaint.yvrtz.workers.dev · Sumber dokumen ini: [docs/dokumentasi-teknis.html](#) (render ulang dengan `npm run docs`).